

Space-Invaders

Definiere jede Klasse in einer eigenen Datei. Der Name der Datei soll immer der Name der Klasse sein.

1 V2

1.1 Aufgabe

Erstelle ein Record V2. Die Objekte dieser Klasse sind Vektoren im zweidimensionalen Raum mit ganzzahligen Komponenten.

```
new V2(3, 2);
```

V2[x=3, y=2]

```
new V2(1, -5);
```

V2[x=1, y=-5]

1.2 Aufgabe

Erweitere die Klasse V2 um die Methode plus. Der Methode wird ein Vektor übergeben. Es wird die Summe der beiden Vektoren zurückgegeben.

```
new V2(3, 2).plus(new V2(1, -5));
```

V2[x=4, y=-3]

```
new V2(3, 4).plus(new V2(-4, -5));
```

V2[x=-1, y=-1]

Hinweis:

$$\begin{pmatrix} v_1 \\ v_2 \end{pmatrix} + \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} v_1 + w_1 \\ v_2 + w_2 \end{pmatrix}$$

2 Utils

2.1 Aufgabe

Lege die Klasse `Utils` in `Utils.java` an. Alle Methoden dieser Klasse sind statisch.

2.2 Aufgabe

Implementiere eine Methode `getXCoordinates`. Der Methode wird eine Liste von `V2` übergeben. Sie gibt eine Liste aller x-Koordinaten zurück.

```
Utils.getXCoordinates(List.of(new V2(3, 4), new V2(7, 9), new V2(3, 5)));
```

[3, 7, 3]

```
Utils.getXCoordinates(List.of(new V2(-1, 2)));
```

[-1]

2.3 Aufgabe

Implementiere eine Methode `getYCoordinates`. Der Methode wird eine Liste von `V2` übergeben. Sie gibt eine Liste aller y-Koordinaten zurück.

```
Utils.getYCoordinates(List.of(new V2(3, 4), new V2(7, 9), new V2(3, 5)));
```

[4, 9, 5]

```
Utils.getYCoordinates(List.of(new V2(5, 2), new V2(-2, 9)));
```

[2, 9]

2.4 Aufgabe

Implementiere eine Methode `isOnBoard` für eine einzelne Zelle. Der Methode werden

- eine Zelle als `V2`,
- die Anzahl der Spalten des Spielfelds und
- die Anzahl der Zeilen des Spielfelds

übergeben. Sie gibt zurück, ob die Zelle auf dem Spielfeld ist. Das ist nur der Fall, wenn

- Keine Koordinate negativ ist und
- Die x-Koordinate echt kleiner ist als die Anzahl der Spalten und
- Die y-Koordinate echt kleiner ist als die Anzahl der Zeilen.

```
Utils.isOnBoard(new V2(3, 4), 10, 8);
```

true

```
Utils.isOnBoard(new V2(-1, 0), 10, 8);
```

false

```
Utils.isOnBoard(new V2(9, 7), 10, 8);
```

true

2.5 Aufgabe

Implementiere eine Methode `isOnBoard` für eine Hitbox. Der Methode werden

- eine `List<V2>` (die Hitbox),
- die Anzahl der Spalten des Spielfelds und
- die Anzahl der Zeilen des Spielfelds

übergeben. Sie gibt `true` zurück, wenn mindestens eine der Zellen in der Liste auf dem Spielfeld liegt.

```
Utils.isOnBoard(List.of(new V2(9,7), new V2(10, 7)), 10, 8);
```

true

```
Utils.isOnBoard(List.of(new V2(-1,7), new V2(0, 7)), 10, 8);
```

true

```
Utils.isOnBoard(List.of(new V2(11,7), new V2(10, 7)), 10, 8);
```

false

Hinweis: Nutze die vorherige `isOnBoard`-Variante für einzelne Zellen.

2.6 Aufgabe

Implementiere eine Methode `intersect`. Der Methode werden zwei Listen vom selben Typ übergeben. Sie gibt `true` zurück, wenn es mindestens ein gemeinsames Element gibt, sonst `false`.

```
Utils.intersect(List.of(1, 2), List.of(2, 3, 4));
```

true

```
Utils.intersect(List.of(new V2(1, 1)),  
                List.of(new V2(1, 5)));
```

false

Hinweis: Nutze contains!.

2.7 Hinweis

Importiere mit

```
import java.util.Random;
```

die Klasse Random! Diese hat eine Methode nextInt. Der Methode wird ein Integer n übergeben. Sie gibt einen zufälligen Integer zurück, der echt kleiner als n ist.

```
var r = new Random();  
r.nextInt(5)
```

0

```
r.nextInt(5)
```

4

2.8 Aufgabe

Implementiere eine statische Methode random.

Der Methode wird eine beliebige Liste übergeben. Die Methode gibt **ein zufälliges Element** aus der Liste zurück.

- Ist die Liste **leer**, soll null zurückgegeben werden.
- Andernfalls soll **genau ein Element** der Liste zufällig ausgewählt werden.

```
Utils.random(List.of(1, 2, 3));
```

2

```
Utils.random(List.of("a", "b", "c"));
```

a

```
Utils.random(List.of());
```

2.9 Aufgabe

Implementiere eine statische Methode `charToV2`.

Der Methode wird ein Zeichen (`char`) übergeben. Die Methode gibt einen Richtungsvektor vom Typ `V2` zurück.

Die Zuordnung ist:

- 'a' → Bewegung nach links
- 'd' → Bewegung nach rechts
- jedes andere Zeichen → keine Bewegung

```
Utils.charToV2('a');
```

V2[x=-1, y=0]

```
Utils.charToV2('d');
```

V2[x=1, y=0]

```
Utils.charToV2('x');
```

V2[x=0, y=0]

3 StringWithLocation

3.1 Aufgabe

Erstelle ein Record `StringWithLocation`. Die Eigenschaften sind

- `string` mit dem Typ `String`
- `location` mit dem Typ `V2`

```
new StringWithLocation("Hi", new V2(3, 4));
```

`StringWithLocation[string=Hi, location=V2[x=3, y=4]]`

```
new StringWithLocation("bye", new V2(5, 7));
```

StringWithLocation[string=bye, location=V2[x=5, y=7]]

4 BasicGameObject

4.1 Aufgabe

Erstelle ein record BasicGameObject! Die Eigenschaften sind

- pos mit dem Typ V2
- displayString mit dem Typ String

```
new BasicGameObject(new V2(3, 4), "xy");
```

BasicGameObject[pos=V2[x=3, y=4], displayString=xy]

```
new BasicGameObject(new V2(10, 5), "abc\ndef");
```

BasicGameObject[pos=V2[x=10, y=5], displayString=abc
def]

4.2 Hinweis

Du kannst die Methoden `lines` und `toList` nutzen, um einen String an den Zeilenumbrüchen (`\n`) aufzuteilen.

```
"abc\ndef\nghi".lines().toList()
```

[abc, def, ghi]

4.3 Aufgabe

Erweitere BasicGameObject um eine Methode `show`.

Sie gibt eine `List<StringWithLocation>` zurück.

Jede Zeile des `displayString` wird in ein `StringWithLocation` umgewandelt.

Die erste Zeile beginnt an der Position des Objekts. Jede weitere Zeile ist eine Zeile weiter unten.

```
var bgol = new BasicGameObject(new V2(3, 4), "xy");  
bgol.show();
```

[StringWithLocation[string=xy, location=V2[x=3, y=4]]]

```
var bgo2 = new BasicGameObject(new V2(10, 5), "abc\ndef");
bgo2.show();
```

```
[StringWithLocation[string=abc, location=V2[x=10, y=5]], StringWithLocation[string=def, location=V2[x=10, y=6]]]
```

4.4 Aufgabe

Erweitere `BasicGameObject` um eine Methode `hitBox`.

Sie gibt eine `List<V2>` zurück, die alle Zellen enthält, auf denen das Objekt gezeichnet wird.

Jedes Zeichen des `displayString` (Zeile für Zeile) belegt genau eine Zelle.

```
var bgo1 = new BasicGameObject(new V2(3, 4), "xy");
bgo1.hitBox();
```

```
[V2[x=3, y=4], V2[x=4, y=4]]
```

```
var bgo2 = new BasicGameObject(new V2(10, 5), "abc\ndef");
bgo2.hitBox();
```

```
[V2[x=10, y=5], V2[x=11, y=5], V2[x=12, y=5], V2[x=10, y=6], V2[x=11, y=6], V2[x=12, y=6]]
```

5 MovableGameObject

5.1 Aufgabe

Lege das record `MovableGameObject` an. Die einzige Eigenschaft hat den Typ `BasicGameObject`.

```
var bgo = new BasicGameObject(new V2(3, 4), "xy");
var mgo = new MovableGameObject(bgo);
mgo;
```

```
MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=4], displayString=xy]]
```

5.2 Aufgabe

Schreibe einen zusätzlichen Konstruktor für `MovableGameObject`. Dem Konstruktor werden eine Position als `V2` und der `displayString` übergeben. Er erzeugt intern ein `BasicGameObject` mit diesen Werten.

```
new MovableGameObject(new V2(3, 4), "xy");
```

```
MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=4], displayString=xy]]
```

```
new MovableGameObject(new V2(10, 5), "abc\ndef");
```

```
MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=10, y=5], displayString=abc def]]
```

Hinweis: Die folgenden 3 Methoden sind nur dazu da um die Methoden der Klasse `BasicGameObject` auch mit einem Objekt der Klasse `MovableGameObject` nutzen zu können. Die Implementierung besteht immer nur aus einer Zeile, in der die entsprechende Methode der Klasse `BasicGameObject` aufgerufen wird.

5.3 Aufgabe

Erweitere `MovableGameObject` um eine Methode `pos`. Sie gibt die Position des gekapselten `BasicGameObject` zurück.

```
var mgo1 = new MovableGameObject(new V2(3, 4), "xy");  
mgo1.pos();
```

```
V2[x=3, y=4]
```

5.4 Aufgabe

Erweitere `MovableGameObject` um eine Methode `show`!

```
var mgo2 = new MovableGameObject(new V2(10, 5), "abc\\ndef");  
mgo2.show();
```

```
[StringWithLocation[string=abc, location=V2[x=10, y=5]], StringWithLocation[string=def, location=V2[x=10, y=6]]]
```

5.5 Aufgabe

Erweitere `MovableGameObject` um eine Methode `hitBox`.

```
var mgo2 = new MovableGameObject(new V2(10, 5), "abc\\ndef");  
mgo2.hitBox();
```

```
[V2[x=10, y=5], V2[x=11, y=5], V2[x=12, y=5], V2[x=10, y=6], V2[x=11, y=6], V2[x=12, y=6]]
```

5.6 Aufgabe

Erweitere `MovableGameObject` um eine Methode `move`. Der Methode wird ein Vektor übergeben, mit dem das Objekt verschoben werden soll. Sie gibt das `MovableGameObject` mit verschobener Position zurück.

```
var mgo1 = new MovableGameObject(new V2(3, 4), "xy");  
mgo1.move(new V2(2, 1));
```

```
MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=5, y=5], displayString=xy]]
```

Hinweis: Nutze die Vektoraddition!

5.7 Aufgabe

Erweitere `MovableGameObject` um eine Methode `touchesLeftBorder`. Sie gibt zurück, ob eine Zelle der Hitbox die x -Koordinate 0 hat.

```
var mgo1 = new MovableGameObject(new V2(0, 4), "xy");
mgo1.touchesLeftBorder();
```

true

```
var mgo2 = new MovableGameObject(new V2(10, 5), "abc\ndef");
mgo2.touchesLeftBorder();
```

false

Hinweis: Nutze `getXCoordinates`, `hitBox` und `contains`!

5.8 Aufgabe

Erweitere `MovableGameObject` um eine Methode `touchesRightBorder`. Der Methode wird die Anzahl der Spalten des Spielfelds übergeben. Sie gibt zurück, ob eine Zelle der Hitbox in der letzten Spalte ist (`width - 1`).

```
var mgo1 = new MovableGameObject(new V2(0, 4), "xy");
mgo1.touchesRightBorder(5);
```

false

```
var mgo1 = new MovableGameObject(new V2(0, 4), "xy");
mgo1.touchesRightBorder(6);
```

false

Hinweis: Nutze `getXCoordinates`, `hitBox` und `contains`!

6 PlayerRocket

6.1 Aufgabe

Definiere ein record `PlayerRocket`. Die einzige Eigenschaft hat den Typ `MovableGameObject`.

```
var mgo = new MovableGameObject(new V2(5, 10), "abc\ndef");
var pr = new PlayerRocket(mgo);
pr;
```

`PlayerRocket[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=5, y=10], displayString=abc\ndef]]]`

6.2 Aufgabe

Implementiere einen zusätzlichen Konstruktor. Dem Konstruktor wird eine Position als V2 übergeben. Er soll intern ein neues MovableGameObject mit dieser Position und dem Display-String |\n^ erzeugen.

```
new PlayerRocket(new V2(10, 5));
```

```
PlayerRocket[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=10, y=5], displayString=|\n^]]]
```

6.3 Aufgabe

Implementiere die Methode isPlayerRocket. Die Methode gibt immer true zurück!

```
var pr = new PlayerRocket(new V2(10, 5));  
pr.isPlayerRocket();
```

```
true
```

Hinweis: Die folgenden 4 Methoden sind nur dazu da um die Methoden der Klasse MovableGameObject auch mit einem Objekt der Klasse PlayerRocket nutzen zu können. Die Implementierung besteht immer nur aus einer Zeile, in der die entsprechende Methode der Klasse MovableGameObject aufgerufen wird.

6.4 Aufgabe

Implementiere die Methode pos. Diese gibt die Position als V2 zurück!

```
var pr = new PlayerRocket(new V2(10, 5));  
pr.pos();
```

```
V2[x=10, y=5]
```

6.5 Aufgabe

Implementiere die Methode show.

```
var pr = new PlayerRocket(new V2(10, 5));  
pr.show();
```

```
[StringWithLocation[string=|, location=V2[x=10, y=5]], StringWithLocation[string=^, location=V2[x=10, y=6]]]
```

6.6 Aufgabe

Implementiere die Methode hitBox.

```
var pr = new PlayerRocket(new V2(10, 5));
```

6.7 Aufgabe

Implementiere die Methode move so, dass sie eine um eine Zeile nach oben verschobene PlayerRocket zurückgibt.

```
var pr = new PlayerRocket(new V2(10, 5));
pr.move();
```

```
PlayerRocket[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=10, y=4], displayString=|
^]]]
```

7 AlienRocket

7.1 Aufgabe

Erstelle ein record AlienRocket. Die einzige Eigenschaft hat den Typ MovableGameObject!

```
var mgo = new MovableGameObject(new V2(3, 4), "xy");
new AlienRocket(mgo);
```

```
AlienRocket[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=4], displayString=xy]]]
```

7.2 Aufgabe

Implementiere einen Konstruktor für AlienRocket, dem nur eine Position als V2 übergeben wird. Er soll intern ein MovableGameObject mit der übergebenen Position und dem String "| \n~" anlegen.

```
new AlienRocket(new V2(3, 4));
```

```
AlienRocket[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=4], displayString=|
~]]]
```

7.3 Aufgabe

Implementiere die Methode isPlayerRocket. Die Methode gibt immer false zurück.

```
var ar = new AlienRocket(new V2(3, 4));
ar.isPlayerRocket();
```

false

Hinweis: Die folgenden 3 Methoden sind nur dazu da um die Methoden der Klasse MovableGameObject auch mit einem Objekt der Klasse AlienRocket nutzen zu können. Die Implementierung besteht immer nur aus einer Zeile, in der die entsprechende Methode der Klasse MovableGameObject aufgerufen wird.

7.4 Aufgabe

Implementiere die Methode pos. Diese gibt die Position zurück.

```
var ar = new AlienRocket(new V2(3, 4));
ar.pos();
```

V2[x=3, y=4]

7.5 Aufgabe

Implementiere die Methode show()!

```
var ar = new AlienRocket(new V2(3, 4));
ar.show();
```

[StringWithLocation[string=|, location=V2[x=3, y=4]], StringWithLocation[string=^, location=V2[x=3, y=5]]]

7.6 Aufgabe

Implementiere die Methode hitBox!

```
var ar = new AlienRocket(new V2(3, 4));
ar.hitBox();
```

[V2[x=3, y=4], V2[x=3, y=5]]

7.7 Aufgabe

Implementiere die Methode move. Diese gibt eine um eine Zeile nach unten verschobene AlienRocket zurück.

```
AlienRocket ar = new AlienRocket(new V2(3,4));
ar.move();
```

AlienRocket[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=5], displayString=|
^]]]

8 Player

8.1 Aufgabe

Erstelle ein record Player. Die einzige Eigenschaft hat den Typ MovableGameObject!

```
new Player(new MovableGameObject(new V2(3,2), "xy"));
```

Player[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=2], displayString=xy]]

8.2 Aufgabe

Implementiere einen zusätzlichen Konstruktor. Dem Konstruktor wird eine Position als V2 übergeben. Er soll intern ein neues MovableGameObject mit dieser Position und dem Display-String "_\MM_nqWAAWp" erzeugen und setzen.

```
new Player(new V2(10, 50));
```

```
Player[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=10, y=50], displayString=_/MM\_  
qWAAWp]]]
```

Hinweis: Die folgenden 4 Methoden sind nur dazu da um die Methoden der Klasse `MovableGameObject` auch mit einem Objekt der Klasse `Player` nutzen zu können. Die Implementierung besteht immer nur aus einer Zeile, in der die entsprechende Methode der Klasse `Player` aufgerufen wird.

8.3 Aufgabe

Implementiere die Methode `pos`. Diese gibt die Position zurück!

```
var player = new Player(new V2(10,5));  
player.pos();
```

V2[x=10, y=5]

8.4 Aufgabe

Implementiere die Methode `show`.

```
var player = new Player(new V2(10,5));  
player.show();
```

```
[StringWithLocation[string=_/MM\_, location=V2[x=10, y=5]], StringWithLocation[string=qWAAWp, location=V2[x=10, y=6]]]
```

8.5 Aufgabe

Implementiere die Methode `hitBox`.

```
var player = new Player(new V2(10,5));  
player.hitBox();
```

```
[V2[x=10, y=5], V2[x=11, y=5], V2[x=12, y=5], V2[x=13, y=5], V2[x=14, y=5], V2[x=15, y=5], V2[x=10, y=6], V2[x=11, y=6], V2[x=12, y=6], V2[x=13, y=6], V2[x=14, y=6], V2[x=15, y=6]]
```

8.6 Aufgabe

Implementiere die Methode `move(V2 dir)`! Diese gibt ein neues Objekt vom Typ `Player` zurück. Dieses wird mit dem übergebenen Vektor verschoben!

```
var player = new Player(new V2(10,5));  
player.move(new V2(1,0));
```

```
Player[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=11, y=5], displayString=_/MM\_  
qWAAWp]]]
```

8.7 Aufgabe

Implementiere die Methode `reactToBorder`. Der Methode wird die Breite des Spielfelds übergeben. Sie gibt ein neues Objekt vom Typ `Player` zurück. Wenn das Objekt rechts von der rechten Grenze ist, wird es um eine Einheit nach links verschoben.

```
var player = new Player(new V2(10,5));
player.reactToBorder(10);
```

```
Player[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=10, y=5], displayString=_/MM\_
qWAAWp]]]
```

Wenn das Objekt links von der linken Grenze ist, wird es um eine Einheit nach rechts verschoben.

```
var player = new Player(new V2(-1,5));
player.reactToBorder(10);
```

```
Player[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=5], displayString=_/MM\_
qWAAWp]]]
```

Ansonsten wird die Position unverändert zurückgegeben.

```
var player = new Player(new V2(3,5));
player.reactToBorder(10);
```

```
Player[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=5], displayString=_/MM\_
qWAAWp]]]
```

Hinweis: Nutze die Methoden `touchesLeftBorder` und `touchesRightBorder` der Klasse `MovableGameObject`!

8.8 Aufgabe

Implementiere die Methode `moveBounded(V2 dir, int width)` so, dass sie ein Objekt vom Typ `Player` zurückgibt. Sie bewegt das Objekt um den Vektor `dir` und passt es anschließend mit `reactToBorder` an die Breite `width` an!

```
new Player(new V2(10,5)).moveBounded(new V2(1,0), 10);
```

```
Player[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=11, y=5], displayString=_/MM\_
qWAAWp]]]
```

Hinweis: Nutze `move` und `reactToBorder`!

8.9 Aufgabe

Implementiere die Methode `shoot`. Diese erzeugt ein neues `PlayerRocket` und gibt es zurück. Es befindet sich zwei Zeilen über dem `Player`-Objekt.

```
new Player(new V2(10,5)).shoot();
```

```
PlayerRocket[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=10, y=3], displayString=|
^]]]
```

9 Alien

9.1 Aufgabe

Lege das record Alien an. Die einzige Eigenschaft hat den Typ `MovableGameObject`.

```
var mgo = new MovableGameObject(new V2(20, 4), "abc\ndef");
var alien = new Alien(mgo);
```

9.2 Aufgabe

Schreibe einen Konstruktor. Dem Konstruktor werden eine Position als `V2` und der `displayString` übergeben. Er erzeugt intern ein `MovableGameObject` mit diesen Werten.

```
new Alien(new V2(20, 4), "abc\ndef");
```

```
Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=4], displayString=abc\ndef]]]
```

Hinweis: Die folgenden Methoden sind nur dazu da um die Methoden der Klasse `MovableGameObject` auch mit einem Objekt der Klasse `Alien` nutzen zu können. Die Implementierung besteht immer nur aus einer Zeile, in der die entsprechende Methode der Klasse `BasicGameObject` aufgerufen wird.

9.3 Aufgabe

Erweitere die Klasse `Alien` um die Methoden `pos`, `show` und `hitBox`. Rufe jeweils die entsprechenden Methoden von `MovableGameObject` auf!

```
var alien = new Alien(new V2(5, 6), "abc\ndef");
alien.pos();
```

```
V2[x=5, y=6]
```

```
alien.show();
```

```
[StringWithLocation[string=abc, location=V2[x=5, y=6]], StringWithLocation[string=def, location=V2[x=5, y=7]]]
```

```
alien.hitBox();
```

```
[V2[x=5, y=6], V2[x=6, y=6], V2[x=7, y=6], V2[x=5, y=7], V2[x=6, y=7], V2[x=7, y=7]]
```

9.4 Aufgabe

Erweitere `Alien` um eine Methode `move`. Der Methode wird ein Vektor übergeben, mit dem das Objekt verschoben werden soll. Sie gibt das `Alien` mit verschobener Position zurück.

```
var alien = new Alien(new V2(5, 6), "abc\ndef");
alien.move(new V2(1, 0));
```

```
Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=6, y=6], displayString=abc\ndef]]]
```

```
alien.move(new V2(-1, 0));
```

```
Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=6], displayString=abc\ndef]]]
```

9.5 Aufgabe

Erweitere Alien um die Methoden `touchesLeftBorder` und `touchesRightBorder(int width)`. Beide Methoden rufen die entsprechenden Methoden der Klasse `MovableGameObject` auf.

```
var alien = new Alien(new V2(5, 6), "abc\ndef");
alien.touchesLeftBorder();
```

false

```
var alien = new Alien(new V2(0, 6), "abc\ndef");
alien.touchesLeftBorder();
```

true

```
var alien = new Alien(new V2(5, 6), "abc\ndef");
alien.touchesRightBorder(8)
```

true

```
var alien = new Alien(new V2(5, 6), "abc\ndef");
alien.touchesRightBorder(9)
```

false

9.6 Aufgabe

Erweitere Alien um eine Methode `isInLastLine`. Der Methode wird die Anzahl der Zeilen des Spielfelds übergeben. Sie bestimmt, ob eine Zelle der Hitbox die letzte spielbare Zeile erreicht.

```
var alien = new Alien(new V2(5, 6), "abc\ndef");
alien.isInLastLine(8);
```

true

```
alien.isInLastLine(9);
```

false

Hinweis: Nutze `Utils.getYCoordinates` und `hitBox`!

9.7 Aufgabe

Implementiere `shoot` in Alien. Es gibt eine `AlienRocket` zurück, die zwei Zeilen unter der aktuellen Position startet.


```
var alien = new Alien(new V2(5, 6), "abc\\ndef");
alien.shoot();
```

```
AlienRocket[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=5, y=8], displayString=|
"]]
```

10 Countdown

10.1 Aufgabe

Lege das record Countdown an. Es hat die Eigenschaften `start` und `current`. Der Typ der beiden Eigenschaften ist `int`.

```
new Countdown(5, 3);
```

```
Countdown[start=5, current=3]
```

10.2 Aufgabe

Schreibe einen zusätzlichen Konstruktor. Dem Konstruktor wird eine Zahl übergeben. Er setzt `current` und `start` auf diese Zahl.

```
var c = new Countdown(7);
c.current();
```

7

```
c.start();
```

7

10.3 Aufgabe

Implementiere die Methode `countDown`. Wenn `current` größer als 0 ist, wird `current` um 1 dekrementiert. Andernfalls wird `current` wieder auf `start` gesetzt.

```
var c = new Countdown(3, 2);
c = c.countDown();
c.current();
```

1

```
c = c.countDown();
c.current();
```

0

```
c = c.countDown();  
c.current();
```

3

10.4 Aufgabe

Implementiere die Methode `finished`. Sie gibt zurück, ob `current` den Wert 0 hat.

```
new Countdown(4, 0).finished();
```

true

```
new Countdown(4, 1).finished();
```

false

11 AlienSwarm

11.1 Aufgabe

Erstelle ein Record `AlienSwarm`. Die Eigenschaften sind `aliensDirection` vom Typ `V2`, `aliens` vom Typ `List<Alien>` und `alienRocketCountdown` vom Typ `CountDown`.

```
new AlienSwarm(new V2(1,0), List.of(), new Countdown(5));
```

```
AlienSwarm[aliensDirection=V2[x=1, y=0], aliens=[], alienRocketCountdown=CountDown[start=5, current=5]]
```

11.2 Aufgabe

Implementiere die statische Methode `rowToAlienStrings`. Der Methode wird eine ganze Zahl zwischen 0 und 2 übergeben. Sie gibt einen `String` zurück, der das Alien in dieser Reihe darstellt.

```
AlienSwarm.rowToAlienStrings(0);
```

```
{00}  
/VV\
```

```
AlienSwarm.rowToAlienStrings(1);
```

```
/MM\  
\~~/
```

```
AlienSwarm.rowToAlienStrings(2);
```

```
{@@}  
/""\
```

11.3 Aufgabe

Implementiere die statische Methode `createAliens`. Diese hat keine Parameter. Sie gibt immer die gleiche `List<Alien>` zurück.

```
AlienSwarm.createAliens();
```

```
[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=0], displayString={@@}  
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=0], displayString={@@}  
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=0], displayString={@@}  
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=0], displayString={@@}  
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=0], displayString={@@}  
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=0], displayString={@@}  
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=0], displayString={@@}  
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=0], displayString={@@}  
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=0], displayString={@@}  
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=0], displayString={@@  
/""\]]]
```

12 Aufgabe

Schreibe einen parameterlosen Konstruktor für `AlienSwarm`. Er initialisiert: - `aliens` mit `createAliens()` - `aliensDirection` mit einem Einheitsvektor nach rechts. - `alien-RocketCountdown` mit einem Countdown mit 5 Sekunden.

```
new AlienSwarm();
```

```
AlienSwarm[aliensDirection=V2[x=1, y=0], aliens=[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=8], displayString={00}  
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=4], displayString=/MM\  
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=4], displayString=/MM\  
\\~/]]]
```

```

\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=4], displayString=/MM\\
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=4], displayString=/MM\\
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=4], displayString=/MM\\
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=4], displayString=/MM\\
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=4], displayString=/MM\\
\\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=0], displayString={@@}
/""\\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=0], displayString={@@}
/""\\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=0], displayString={@@}
/""\\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=0], displayString={@@}
/""\\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=0], displayString={@@}
/""\\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=0], displayString={@@}
/""\\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=0], displayString={@@}
/""\\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=0], displayString={@@}
/""\\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=0], displayString={@@}
/""\\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=0], displayString={@@}
/""\\]], alienRocketCountdown=CountDown[start=5, current=5]]

```

12.1 Aufgabe

Implementiere die Methode `noAliensLeft`. Sie gibt zurück, ob die Aliens-Liste leer ist.

```

new AlienSwarm().noAliensLeft();

false

new AlienSwarm(new V2(1,0), List.of(), new CountDown(5)).noAliensLeft();

true

```

12.2 Aufgabe

Implementiere die Methode `aliensAreInLastLine`. Sie bekommt die Spielfeldhöhe. Sie gibt `true` zurück, wenn ein Alien die letzte spielbare Zeile erreicht.

```

var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\\nb"),
        new Alien(new V2(3, 5), "b\\nc")
    ),
    new CountDown(5));
alienSwarm.aliensAreInLastLine(7);

true

alienSwarm.aliensAreInLastLine(8);

false

```

Hinweis: Nutze die Methode `isInLastLine` der Klasse `Alien`!

12.3 Aufgabe

Implementiere die Methode `move`. Sie verschiebt alle Aliens um den Vektor `aliensDirection` und reduziert den `Countdown`.

```

var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\\nb"),
        new Alien(new V2(3, 5), "b\\nc")
    ),
    new CountDown(5));
alienSwarm.move();

```

```

AlienSwarm[aliensDirection=V2[x=1, y=0], aliens=[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=5], displayString=a
b]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=5], displayString=b
c]]], alienRocketCountdown=CountDown[start=5, current=4]]

```

```
var alienSwarm = new AlienSwarm(new V2(0,1),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.move();
```

AlienSwarm[aliensDirection=V2[x=0, y=1], aliens=[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=2, y=6], displayString=a b]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=6], displayString=b c]]], alienRocketCountdown=CountDown[start=5, current=4]]

Hinweis: Nutze die Methode `move` der Klasse `Alien`!

12.4 Aufgabe

Implementiere die Methode `touchesLeftBorder`. Sie gibt `true` zurück, wenn ein Alien die linke Grenze berührt.

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.touchesLeftBorder();
```

false

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(0, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.touchesLeftBorder();
```

true

Hinweis: Nutze die Methode `touchesLeftBorder` der Klasse `Alien`!

12.5 Aufgabe

Implementiere die Methode `touchesRightBorder`. Sie bekommt die Spielfeldbreite übergeben. Sie gibt `true` zurück, wenn ein Alien die rechte Grenze berührt.

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.touchesRightBorder(4);
```

true

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.touchesRightBorder(5);
```

false

Hinweis: Nutze die Methode `touchesRightBorder` der Klasse `Alien`!

12.6 Aufgabe

Implementiere die Methode `computeNextAlienDirection`. Sie bekommt die Spielfeldbreite übergeben. Sie berechnet die nächste Richtung der Aliens.

Wenn sich die Aliens nach rechts bewegen und den rechten Rand berühren, ist die nächste Richtung nach unten.

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.computeNextAlienDirection(3)
```

V2[x=0, y=1]

Wenn sich die Aliens nach links bewegen und den linken Rand berühren, ist die nächste Richtung nach unten.

```
var alienSwarm = new AlienSwarm(new V2(-1, 0),
    List.of(
        new Alien(new V2(0, 5), "a\nb"),
        new Alien(new V2(1, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.computeNextAlienDirection(4)
```

V2[x=0, y=1]

Wenn sich die Aliens nach unten bewegen und den rechten Rand berühren, ist die nächste Richtung nach links.

```
var alienSwarm = new AlienSwarm(new V2(0, 1),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.computeNextAlienDirection(4)
```

V2[x=-1, y=0]

Wenn sich die Aliens nach unten bewegen und den linken Rand berühren, ist die nächste Richtung nach rechts.

```
var alienSwarm = new AlienSwarm(new V2(0, 1),
    List.of(
        new Alien(new V2(0, 5), "a\nb"),
        new Alien(new V2(1, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.computeNextAlienDirection(4)
```

V2[x=1, y=0]

Ansonsten wird die Richtung nicht geändert.

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.computeNextAlienDirection(5)
```

V2[x=1, y=0]

12.7 Aufgabe

Implementiere die Methode `reactToBorder`. Sie nutzt `computeNextAlienDirection` und gibt ein neues `AlienSwarm` mit aktualisierter Richtung zurück.

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.reactToBorder(4);
```

```
AlienSwarm[aliensDirection=V2[x=0, y=1], aliens=[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=2, y=5], displayName=a
b]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=5], displayName=b
c]]], alienRocketCountdown=Countdown[start=5, current=5]]
```

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.reactToBorder(5);
```

```
AlienSwarm[aliensDirection=V2[x=1, y=0], aliens=[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=2, y=5], displayName=a
b]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=5], displayName=b
c]]], alienRocketCountdown=Countdown[start=5, current=5]]
```

12.8 Aufgabe

Implementiere die Methode `moveBounded`. Sie bewegt die Aliens und ändert eventuell die Richtung.

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(4));
alienSwarm.moveBounded(5);
```

```
AlienSwarm[aliensDirection=V2[x=0, y=1], aliens=[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=5], displayName=a
b]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=5], displayName=b
c]]], alienRocketCountdown=Countdown[start=4, current=3]]
```

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.moveBounded(6);
```

```
AlienSwarm[aliensDirection=V2[x=1, y=0], aliens=[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=5], displayName=a
b]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=5], displayName=b
c]]], alienRocketCountdown=Countdown[start=5, current=4]]
```

Hinweis: Nutze `move` und `reactToBorder` !

12.9 Aufgabe

Implementiere die Methode `countDownFinished`. Sie gibt `true` zurück, wenn der Countdown fertig ist.

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5));
alienSwarm.countDownFinished();
```

false

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5, 0));
alienSwarm.countDownFinished();
```

true

12.10 Aufgabe

Implementiere die Methode `getLowestAliens`. Sie gibt alle Aliens aus der niedrigsten Zeile zurück. Annahme: Die Aliens sind so geordnet, dass die niedrigste Zeile zuerst kommt.

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5, 0));
alienSwarm.getLowestAliens();
```

```
[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=2, y=5], displayString=a
b]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=5], displayString=b
c]]]]
```

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 4), "b\nc")
    ),
    new Countdown(5, 0));
alienSwarm.getLowestAliens();
```

```
[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=2, y=5], displayString=a
b]]]]
```

12.11 Aufgabe

Implementiere die Methode `getShootingAlien` in `AlienSwarm`.

- Die Methode gibt ein zufälliges Alien aus der niedrigsten Reihe zurück, das schießen soll.
- Ein Alien darf nur dann schießen, wenn der **Countdown** fertig ist.
- Ist der Countdown **nicht fertig**, soll null zurückgegeben werden.
- Nutze die Methode `getLowestAliens` und die Utility-Methode `Utils.random`.


```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5, 0));
alienSwarm.getShootingAlien();
```

```
Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=3, y=5], displayString=b
c]]]
```

```
var alienSwarm = new AlienSwarm(new V2(1,0),
    List.of(
        new Alien(new V2(2, 5), "a\nb"),
        new Alien(new V2(3, 5), "b\nc")
    ),
    new Countdown(5, 1));
alienSwarm.getShootingAlien();
```

13 LevelFactory

13.1 Aufgabe

Lege die Klasse LevelFactory an.

Diese Klasse dient dazu, **Spielobjekte für Level-Elemente zu erzeugen**.

Alle Methoden dieser Klasse sind **statisch**.

13.2 Aufgabe

Implementiere die statische Methode **generateBlock**.

Der Methode werden übergeben:

- eine Startposition pos vom Typ V2
- die Anzahl der Spalten cols
- die Anzahl der Zeilen rows

Die Methode erzeugt ein **rechteckiges Blockfeld** aus BasicGameObject-Objekten.

- Jedes Objekt hat den Display-String "#"
- Die linke obere Ecke des Blocks beginnt bei pos
- Die Methode gibt eine Liste mit allen erzeugten Blöcken zurück

```
var blocks = LevelFactory.generateBlock(new V2(0, 0), 3, 2);
for (var block: blocks){
    println(block);
}
```

```
BasicGameObject[pos=V2[x=0, y=0], displayString=#]
BasicGameObject[pos=V2[x=1, y=0], displayString=#]
BasicGameObject[pos=V2[x=2, y=0], displayString=#]
BasicGameObject[pos=V2[x=0, y=1], displayString=#]
BasicGameObject[pos=V2[x=1, y=1], displayString=#]
BasicGameObject[pos=V2[x=2, y=1], displayString=#]
```

```
var blocks = LevelFactory.generateBlock(new V2(5, 4), 1, 4);
for (var block: blocks){
    println(block);
}
```

```
BasicGameObject[pos=V2[x=5, y=4], displayString=#]
BasicGameObject[pos=V2[x=5, y=5], displayString=#]
BasicGameObject[pos=V2[x=5, y=6], displayString=#]
BasicGameObject[pos=V2[x=5, y=7], displayString=#]
```

Hinweis:

Nutze verschachtelte for-Schleifen und die Methode plus der Klasse V2.

13.3 Aufgabe

Implementiere die statische Methode generateBlocks.

Der Methode werden übergeben:

- eine Startposition startPos vom Typ V2
- die Anzahl der Spalten cols
- die Anzahl der Zeilen rows
- die Anzahl der Blöcke count

Die Methode erzeugt **mehrere rechteckige Blöcke nebeneinander**.

- Jeder Block wird mit generateBlock erzeugt
- Zwischen zwei Blöcken liegt ein horizontaler Abstand von **2 Spalten**
- Die Methode gibt eine List<BasicGameObject> zurück, die alle Blöcke enthält

```
var blocks = LevelFactory.generateBlocks(new V2(0, 0), 3, 2, 3);
for (var block: blocks){
    println(block);
}
```

```
BasicGameObject[pos=V2[x=0, y=0], displayString=#]
BasicGameObject[pos=V2[x=1, y=0], displayString=#]
BasicGameObject[pos=V2[x=2, y=0], displayString=#]
BasicGameObject[pos=V2[x=0, y=1], displayString=#]
BasicGameObject[pos=V2[x=1, y=1], displayString=#]
BasicGameObject[pos=V2[x=2, y=1], displayString=#]
BasicGameObject[pos=V2[x=5, y=0], displayString=#]
BasicGameObject[pos=V2[x=6, y=0], displayString=#]
BasicGameObject[pos=V2[x=7, y=0], displayString=#]
BasicGameObject[pos=V2[x=5, y=1], displayString=#]
BasicGameObject[pos=V2[x=6, y=1], displayString=#]
BasicGameObject[pos=V2[x=7, y=1], displayString=#]
BasicGameObject[pos=V2[x=10, y=0], displayString=#]
BasicGameObject[pos=V2[x=11, y=0], displayString=#]
BasicGameObject[pos=V2[x=12, y=0], displayString=#]
BasicGameObject[pos=V2[x=10, y=1], displayString=#]
BasicGameObject[pos=V2[x=11, y=1], displayString=#]
BasicGameObject[pos=V2[x=12, y=1], displayString=#]
```

```
var blocks = LevelFactory.generateBlocks(new V2(5, 5), 2, 1, 5);
for (var block: blocks){
    println(block);
}
```

```
BasicGameObject[pos=V2[x=5, y=5], displayString=#]
BasicGameObject[pos=V2[x=6, y=5], displayString=#]
BasicGameObject[pos=V2[x=9, y=5], displayString=#]
BasicGameObject[pos=V2[x=10, y=5], displayString=#]
BasicGameObject[pos=V2[x=13, y=5], displayString=#]
BasicGameObject[pos=V2[x=14, y=5], displayString=#]
BasicGameObject[pos=V2[x=17, y=5], displayString=#]
BasicGameObject[pos=V2[x=18, y=5], displayString=#]
BasicGameObject[pos=V2[x=21, y=5], displayString=#]
BasicGameObject[pos=V2[x=22, y=5], displayString=#]
```

Hinweis:

Nutze generateBlock!

14 Utils

14.1 Aufgabe

Implementiere die statische Methode `repeat`. Der Methode werden ein `String` und eine Zahl `n` übergeben. Sie gibt einen neuen `String` zurück, der den ursprünglichen `String` `n`-mal hintereinander enthält.

```
Utils.repeat("ab", 3);
```

```
ababab
```

```
Utils.repeat("xy\n", 2);
```

```
xy
xy
```

```
Utils.repeat("hi", 0);
```

Hinweis: Nutze eine `for`-Schleife und String-Konkatenation!

15 Plasma

15.1 Aufgabe

Lege das record `Plasma` an. Die einzige Eigenschaft hat den Typ `MovableGameObject`.

```
var mgo = new MovableGameObject(new V2(5, 3), "abc\ndef");
new Plasma(mgo);
```

```
Plasma[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=5, y=3], displayString=abc
def]]]
```

15.2 Aufgabe

Schreibe einen zusätzlichen Konstruktor. Ihm werden eine Position `pos` als `V2` und eine Höhe `height` als `int` übergeben. Er baut den Display-String des Plasmas aus `height` vielen Wiederholungen des Strings `"(((|||||))))\n"` auf.

```
new Plasma(new V2(5, 3), 3);
```

```
Plasma[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=5, y=3], displayString=(((|||||))))
(((|||||))))
(((|||||))))
]]]
```

```
new Plasma(new V2(0, 0), 1);
```

```
Plasma[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=0], displayString=(((|||||))))
]]]
```

Hinweis: Nutze die Methode `Utils.repeat`!

15.3 Aufgabe

Implementiere die Methoden `pos`, `show` und `hitBox`. Rufe jeweils die entsprechenden Methoden von `MovableGameObject` auf.

```
var plasma = new Plasma(new V2(5, 3), 2);
plasma.pos();
```

V2[x=5, y=3]

```
plasma.show();
```

```
[StringWithLocation[string=((|||||))], location=V2[x=5, y=3]], StringWithLocation[string=((|||||))], location=V2[x=5, y=4]]
```

```
plasma.hitBox();
```

```
[V2[x=5, y=3], V2[x=6, y=3], V2[x=7, y=3], V2[x=8, y=3], V2[x=9, y=3], V2[x=10, y=3], V2[x=11, y=3], V2[x=12, y=3], V2[x=13, y=3], V2[x=14, y=3], V2[x=15, y=3], V2[x=16, y=3]]
```

15.4 Aufgabe

Implementiere die Methode `isPlayerRocket`. Sie gibt immer `true` zurück.

```
var plasma = new Plasma(new V2(5, 3), 2);
plasma.isPlayerRocket();
```

true

15.5 Aufgabe

Implementiere die Methode `move`. Sie gibt eine neue Plasma-Rakete zurück, die um 3 Zeilen nach oben verschoben ist.

```
var plasma = new Plasma(new V2(5, 10), 3);
plasma.move().pos();
```

V2[x=5, y=7]

Hinweis: Nutze die Vektoraddition!

16 InvisiblePlasmaCannon

16.1 Aufgabe

Lege das record `InvisiblePlasmaCannon` an. Die einzige Eigenschaft ist `pos` vom Typ `V2`.

```
new InvisiblePlasmaCannon(new V2(10, 15));
```

InvisiblePlasmaCannon[pos=V2[x=10, y=15]]

16.2 Aufgabe

Implementiere die Methode `shoot`. Sie gibt eine Plasma-Rakete zurück. Die Höhe des Plasmas entspricht der y-Koordinate der Kanone. Das Plasma startet direkt über der Kanone.

```
var cannon = new InvisiblePlasmaCannon(new V2(10, 15));
cannon.shoot();
```

```
Plasma[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=10, y=0], displayString=((((((((((((((((
((((((((((((((((
((((((((((((((((
((((((((((((((((
((((((((((((((((
((((((((((((((((
((((((((((((((((
((((((((((((((((
((((((((((((((((
((((((((((((((((
((((((((((((((((
((((((((((((((((
((((((((((((((((
((((((((((((((((
]]]]
```

```
var cannon = new InvisiblePlasmaCannon(new V2(5, 8));
cannon.shoot().pos();
```

```
V2[x=5, y=0]
```

Hinweis: Nutze die Vektoraddition!

17 LevelFactory

17.1 Aufgabe

Implementiere eine weitere statische Methode `generateBlocks`. Ihr werden `width` und `height` des Spielfelds übergeben. Sie erzeugt die Bunker-Blöcke für das Spiel und gibt sie zurück.

- Die Blöcke beginnen bei $x = 1$ und bei $y = 3/4$ der Spielfeldhöhe
- Jeder Block ist 4 Spalten und 3 Zeilen groß
- Der Abstand zwischen zwei Blöcken beträgt 2 Zellen
- Die Anzahl der Blöcke richtet sich nach der Spielfeldbreite

```
LevelFactory.generateBlocks(100, 60);
```

```
[BasicGameObject[pos=V2[x=1, y=45], displayString=#], BasicGameObject[pos=V2[x=2, y=45],
```

Hinweis: Nutze die bereits implementierte Methode `generateBlocks`!

18 Model

18.1 Aufgabe

Lege die Klasse `Model` an. Die Eigenschaften sind:

- `width` und `height` vom Typ `int` (unveränderlich)
- `player` vom Typ `Player`
- `alienSwarm` vom Typ `AlienSwarm`
- `blocks` vom Typ `List<BasicGameObject>`

18.2 Aufgabe

Implementiere die Methode `restart`. Sie setzt alle veränderbaren Eigenschaften auf ihre Anfangswerte zurück:

- `player` startet in der Mitte des Spielfelds, zwei Zeilen über dem unteren Rand
- `alienSwarm` wird neu erzeugt
- `blocks` wird mit `LevelFactory.generateBlocks` erzeugt

```
var model = new Model(100, 60);
model.restart();
println(model.player);
println(model.alienSwarm);
println(model.blocks);
```

```
Player[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=50, y=58], displayString=_/MM\_\_
qWAAWp]]]
AlienSwarm[aliensDirection=V2[x=1, y=0], aliens=[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=0], displayString={@@}
/""\]]], alienRocketCountdown=CountDown[start=5, current=5]]
[BasicGameObject[pos=V2[x=1, y=45], displayString=#], BasicGameObject[pos=V2[x=2, y=45], displayString=#], BasicGameObject[pos=V2[x=3, y=45], displayString=#], Basic
GameObject[pos=V2[x=73, y=47], displayString=#], BasicGameObject[pos=V2[x=74, y=47], displayString=#], BasicGameObject[pos=V2[x=75, y=47], displayString=#], Basic
```

18.3 Aufgabe

Schreibe den Konstruktor. Ihm werden `width` und `height` übergeben. Er setzt die Felder und ruft anschließend `restart` auf.

```
var model = new Model(100, 60);
println(model.width);
println(model.height);
println(model.player);
println(model.alienSwarm);
println(model.blocks);
```

```
100
60
Player[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=50, y=58], displayString=_/MM\_\_
qWAAWp]]]
AlienSwarm[aliensDirection=V2[x=1, y=0], aliens=[Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=8], displayString={00}
/VV\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=4], displayString=/MM\
\~/]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=36, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=32, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=28, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=24, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=20, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=16, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=12, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=8, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=4, y=0], displayString={@@}
/""\]], Alien[mgo=MovableGameObject[basicGameObject=BasicGameObject[pos=V2[x=0, y=0], displayString={@@}
/""\]]], alienRocketCountdown=CountDown[start=5, current=5]]
[BasicGameObject[pos=V2[x=1, y=45], displayString=#], BasicGameObject[pos=V2[x=2, y=45], displayString=#], BasicGameObject[pos=V2[x=3, y=45], displayString=#], Basic
GameObject[pos=V2[x=73, y=47], displayString=#], BasicGameObject[pos=V2[x=74, y=47], displayString=#], BasicGameObject[pos=V2[x=75, y=47], displayString=#], Basic
```


19 TUI

Ergänze in pom.xml die folgende Abhängigkeit:

```
<dependency>
  <groupId>com.googlecode.lanterna</groupId>
  <artifactId>lanterna</artifactId>
  <version>3.1.5</version>
</dependency>
```

19.1 Hinweis

Die Klasse TUI ist bis auf eine Methode bereits vollständig vorgegeben und muss nicht implementiert werden. Sie kümmert sich um die Darstellung des Spiels im Terminal.

Die Klasse hat folgende Methoden:

- `print(List<StringWithLocation> uiState)` — zeichnet den aktuellen Spielzustand
- `printString(String s)` — zeigt einen Text in der Mitte des Terminals an
- `getPressedKey()` — gibt die zuletzt gedrückte Taste zurück
- `close()` — schließt das Terminal

Nur die Methode `print` muss noch implementiert werden.

19.2 Aufgabe

Implementiere die Methode `print`. Ihr wird der aktuelle Spielzustand als `List<StringWithLocation>` übergeben. Sie:

1. löscht den Bildschirm. Hierfür wird auf der Eigenschaft `screen` die Methode `clear` aufgerufen.
2. zeichnet jeden Eintrag der Liste an der richtigen Position. Rufe dafür auf der Eigenschaft `textGraphics` die Methode `putString` auf. Der Methode werden x- und y-Koordinate sowie ein String, der an dieser Stelle ausgegeben werden soll, übergeben.
3. aktualisiert den Bildschirm. Hierfür wird auf der Eigenschaft `screen` die Methode `refresh` aufgerufen.
4. wartet 10 Millisekunden. Dafür wird `sleep(10)` aufgerufen.

```
var tui = new TUI(20, 7);
var xs = List.of(new StringWithLocation("Hello", new V2(5, 3)),
                 new StringWithLocation("World", new V2(9, 5))
               )
tui.print(xs);
```

20 Controller

20.1 Aufgabe

Lege die Klasse `Controller` an. Die Eigenschaften sind:

- `tui` vom Typ `TUI`
- `model` vom Typ `Model`

20.2 Aufgabe

Schreibe einen Konstruktor. Ihm werden `cols` und `rows` übergeben. Er initialisiert `tui` und `model` mit den übergebenen Werten.

```
var controller = new Controller(100, 60);
```

21 Fertigstellung

Um das Spiel fertigzustellen, musst du *Interfaces* verwenden und ein zweites Arbeitsblatt durcharbeiten.